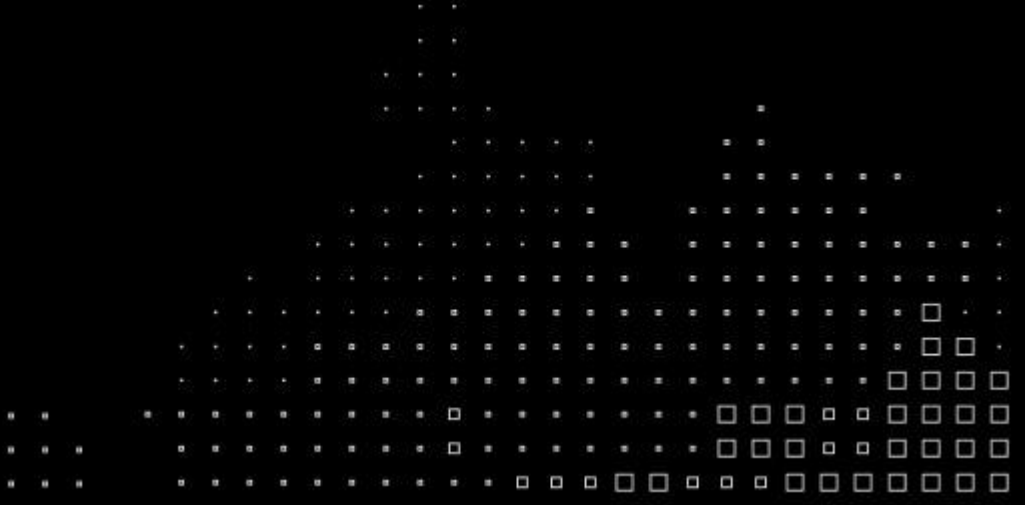
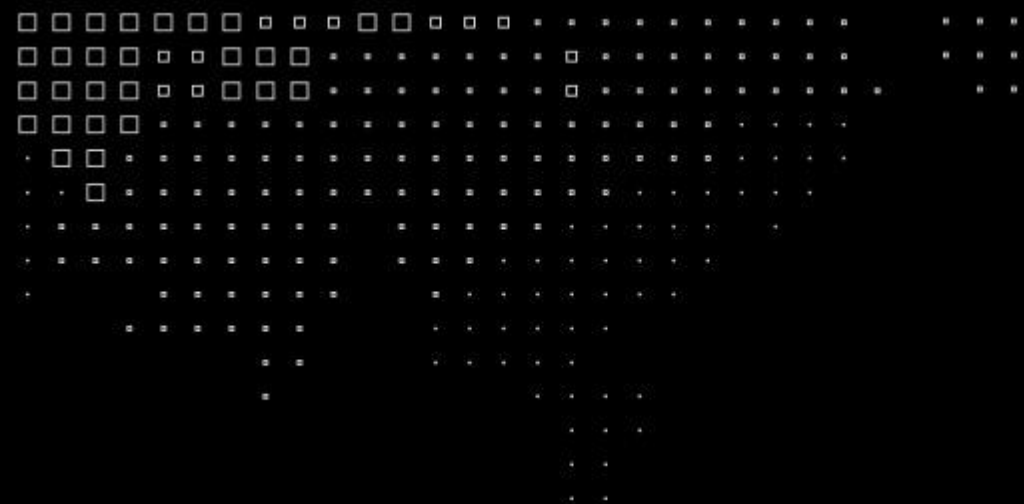


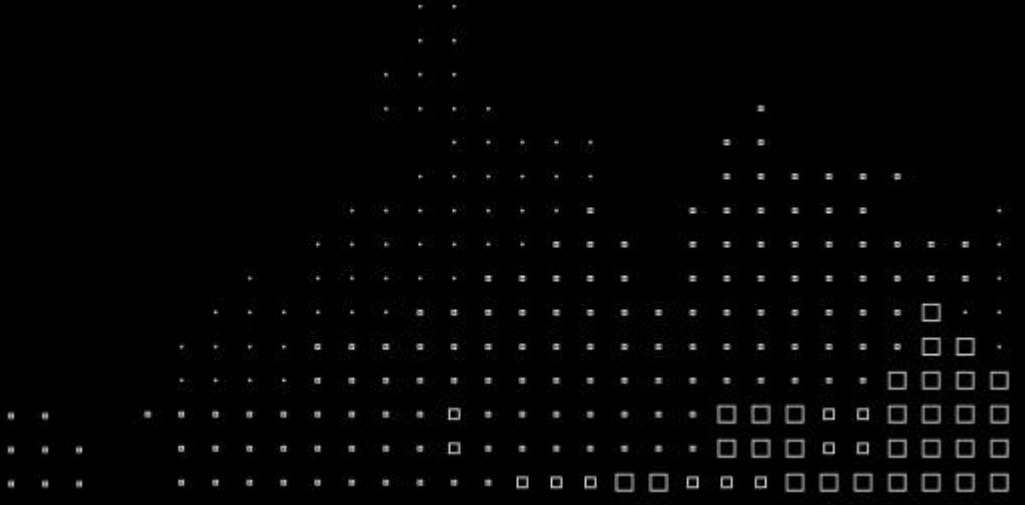
MBA⁺

**Data Science &
Artificial Intelligence**



**MBA⁺****Data Architecture,
Integration and Ingestion**

Prof.: Leandro Mendes
Revisão V2 — parte relacional
(mai/2026)



Comunicação é Repetição!

Recapitulando...

- Definições de arquitetura de dados, bancos de dados e tipos de dados
- Data warehouse (surgimento, aplicações e características)
- Data Lake (Hadoop, processamento distribuído, arquiteturas e ingestão)
- Data Lakehouse (combinação das melhores características anteriores)
- Data Mesh (domínios de dados, dado como produto e proximidade com consumo)

Data Architecture, Integration and Ingestion

(O que vamos explorar?)

Aula 1 – 14/mai (qui)

- Sistemas de Gerenciamento de banco de dados
- Estratégias de arquitetura
- Data Warehouse, Data lake, Lakehouse e Data Mesh

Aula 2 – 28/mai (qui)

- Exemplos de Bancos, diferenças e usos:
 - Bancos Relacionais
 - Bancos Colunares

Aula 3 – 11/Jun (qui)

- Exemplos de Bancos, diferenças e usos:
 - Bancos chave-valor
 - Bancos de documentos
 - Bancos de Grafos & Vector

Aula 4 – 16/jun (ter)

- Ingestão de dados, tratamentos e manipulações
- Pipeline de dados, governança e qualidade
- Integração de dados
 - Cargas batch, ETL, vantagens e desvantagens

Aula 5 – 07/jul (ter)

- Eventos, APIs e casos de uso
- Visualização de dados
- Boas práticas e recomendações

Vamos criar VM na Cloud Azure para os exercícios

Bancos de dados relacionais (SQL)

Por que começar pelo relacional?

FIAP

- ✓ Espinha dorsal transacional (OLTP) da maioria dos sistemas corporativos.
- ✓ Maduro (50+ anos), padronizado (SQL ISO) e com forte consistência (ACID).
- ✓ Base dos fluxos do caso QuantumFinance: **cadastro de cliente** e **transações DOC/TED/PIX**.
- ✓ Dominar modelagem aqui é pré-requisito para depois decidir o que delegar a NoSQL/colunar.

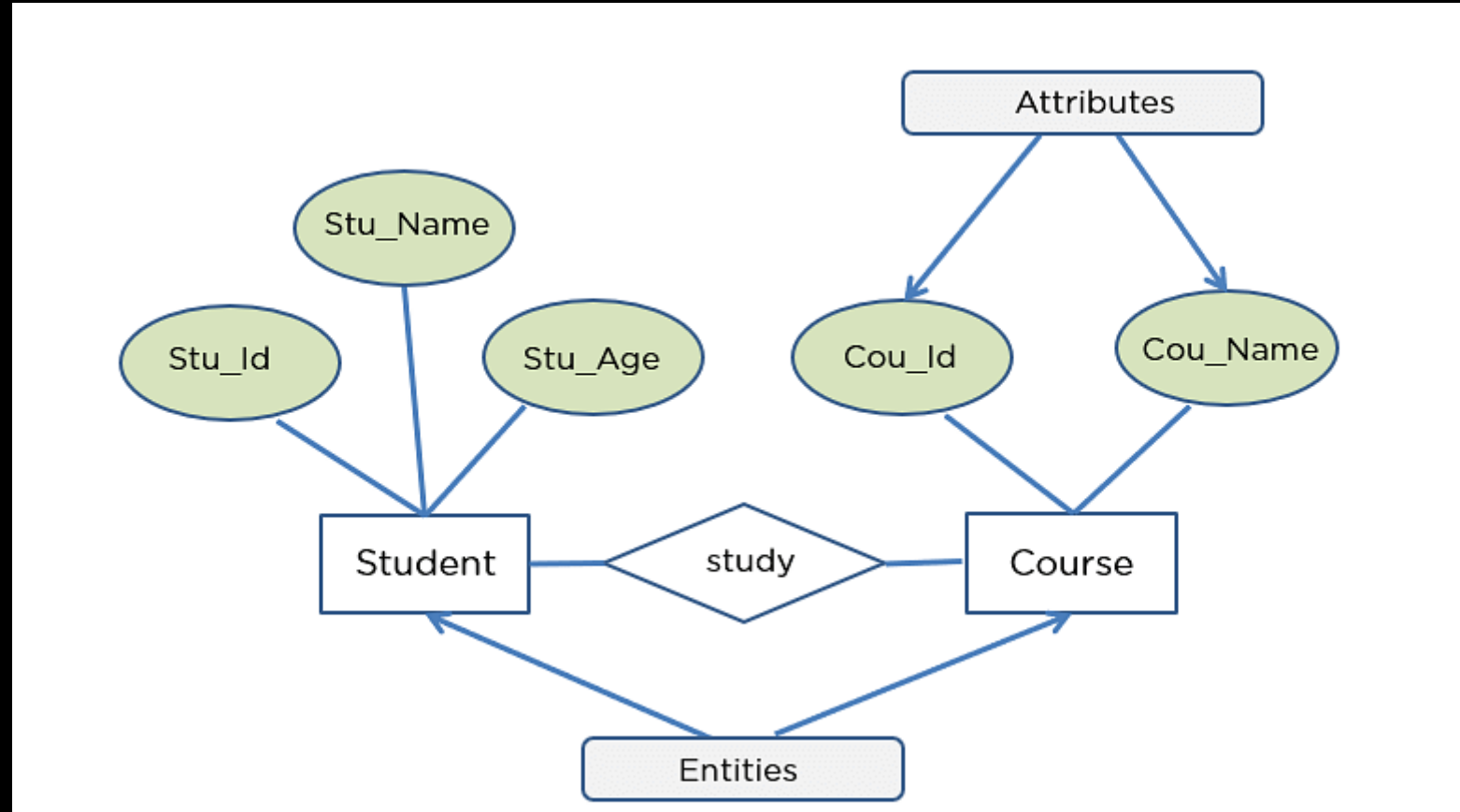
Banco de dados, SGBD e SGBDR

- ✓ Banco de dados: dados inter-relacionados que atendem usuários e aplicações.
- ✓ SGBD: define, carrega, recupera e mantém o banco — isolando a aplicação do armazenamento físico.
- ✓ SGBDR: SGBD **relacional** — tabelas + integridade + SQL.
- ✓ Papéis: DBA (físico/operação) e AD — Administrador de Dados (lógico/modelo).

Características de RDBMS (bancos relacionais)

- ✓ Chave primária: garante a unicidade de cada registro na tabela.
- ✓ Chave estrangeira: relaciona uma tabela à chave primária de outra.
- ✓ Modelo de dados estruturado e pré-definido (schema-on-write).
- ✓ Linguagem padronizada (SQL) para definição, manipulação, controle e transação.
- ✓ Propriedades ACID: **Atomicidade**, **Consistência**, **Isolamento**, **Durabilidade**.

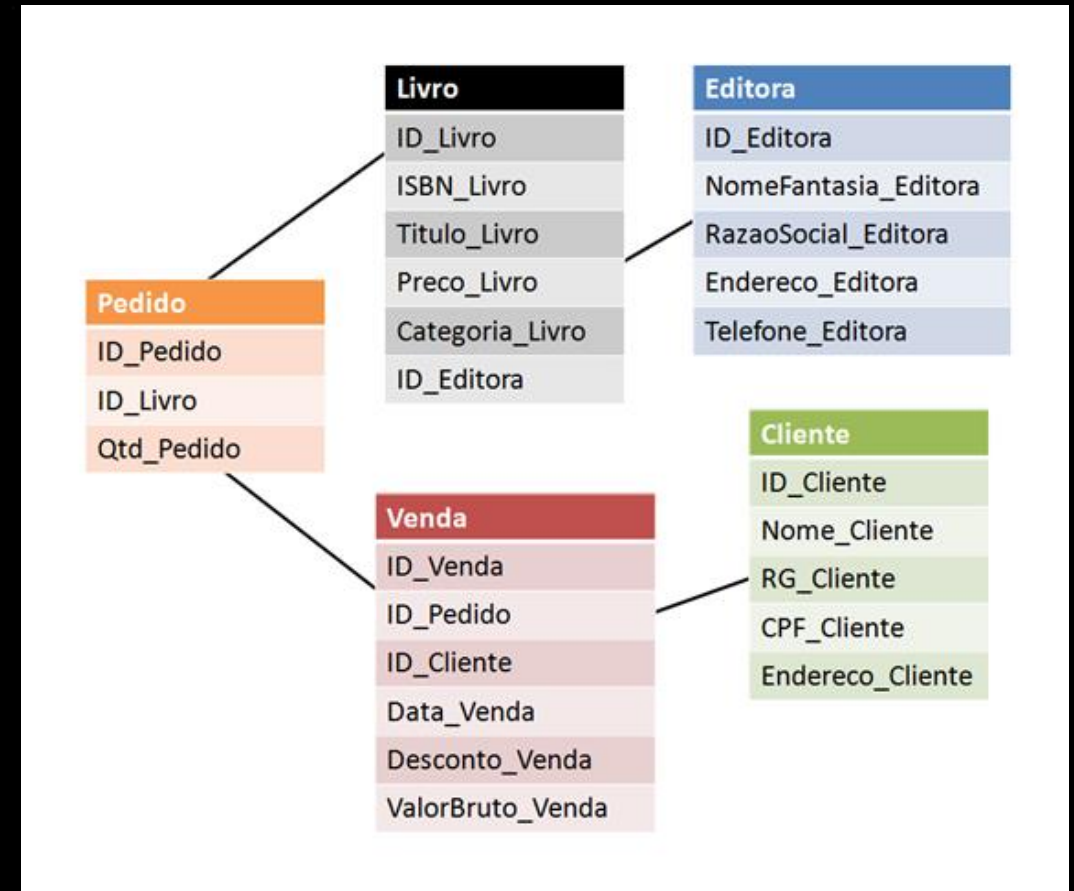
Modelo relacional



45697056

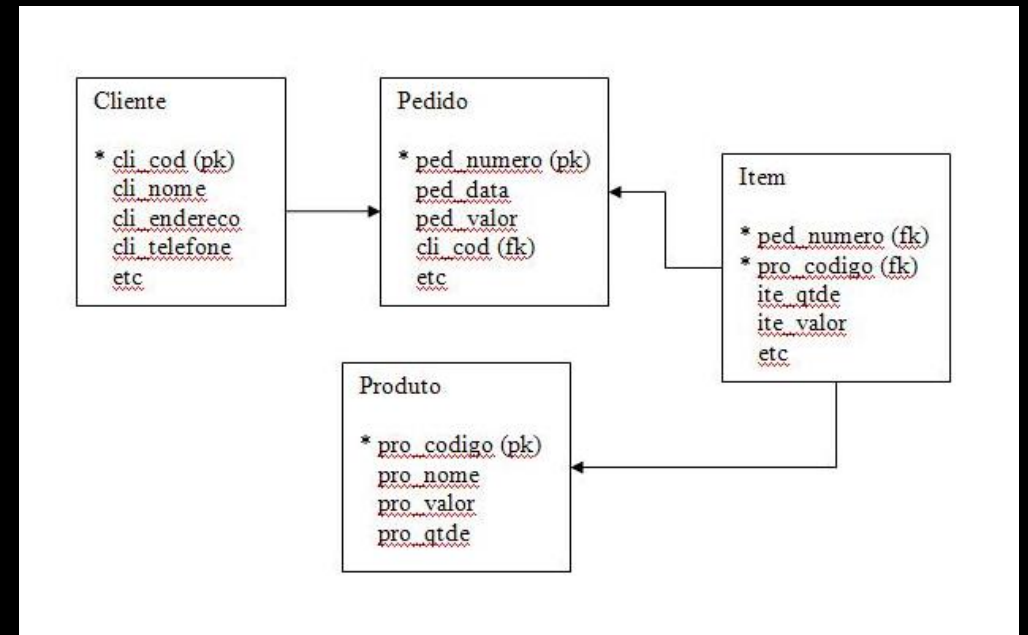
Anatomia de uma relação (tabela)

- ✓ Relação = tabela; **linha** = tupla; **coluna** = atributo.
- ✓ Domínio: conjunto de valores válidos de um atributo.
- ✓ Grau = nº de colunas; Cardinalidade = nº de linhas.
- ✓ Ligações por valores de atributos — não por ponteiros físicos.



Chaves: primária, candidata e artificial

- ✓ **PK:** identifica unicamente cada linha; não admite nulo.
- ✓ **Candidata/natural:** identifica, mas não foi a PK (CPF, e-mail).
- ✓ **Artificial/surrogate:** id gerado (auto-incremento, UUID).
- ✓ **PK composta:** comum em tabelas associativas.



45697056

Integridade de domínio e de entidade

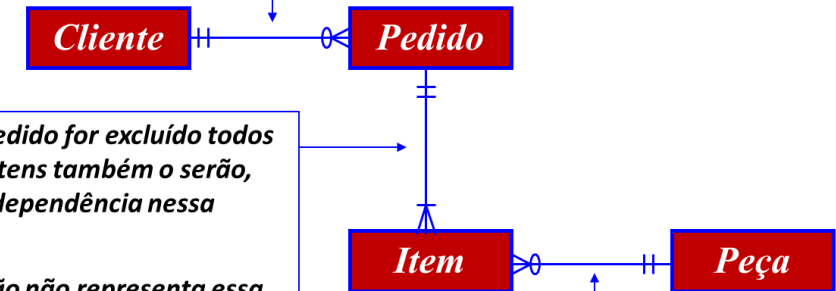
- ✓ Integridade de entidade: toda tabela tem PK e a PK nunca é nula.
- ✓ Integridade de domínio: cada atributo só aceita valores do seu domínio (tipo, faixa, formato).
- ✓ Obrigatoriedade: campos NOT NULL exigidos na inserção/atualização.
- ✓ Restrições: **NOT NULL, UNIQUE, CHECK, DEFAULT**.

Chave estrangeira e integridade referencial

- ✓ **FK:** valores que devem existir na PK de outra tabela.
- ✓ Integridade referencial: o SGBD impede 'órfãos'.
- ✓ Ações ON DELETE/UPDATE: **RESTRICT**, **CASCADE**, **SET NULL**.
- ✓ Tabela referenciada não some (DROP) sem tratar as FKs.

- Aplicando a Notação Pé-de-galinha

Se um Pedido for excluído o cliente a ele associado não o será.
A notação não representa essa situação!



Se um pedido for excluído todos os seus itens também o serão, pois há dependência nessa relação.
A notação não representa essa situação!

Se um Item de Pedido for excluído a peça a ele associado não o será.
A notação não representa essa situação!

Relacionamentos e cardinalidade

- ✓ 1:1 — Cliente $\leftrightarrow$ Perfil de crédito.
- ✓ 1:N — Cliente $\rightarrow$ Contas; Conta $\rightarrow$ Transações.
- ✓ N:N — resolve-se com tabela associativa (Cliente $\leftrightarrow$ Produto).
- ✓ Participação obrigatória ou opcional em cada lado.

- Notação Pé-de-galinha

- Quando um A está associado a um B.



- Quando um A está associado a um ou nenhum B.

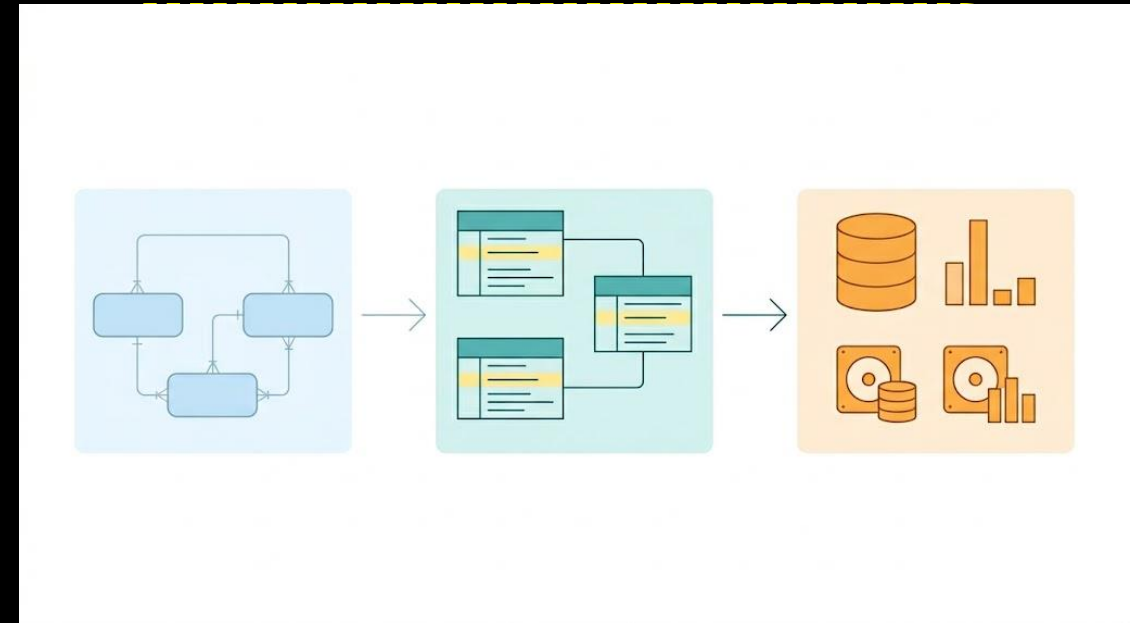


- Quando um A está associado a um, nenhum ou vários B.



Conceitual → Lógico → Físico

- ✓ **Conceitual:** entidades, relacionamentos e regras (modelo ER).
- ✓ **Lógico:** tabelas, atributos atômicos, PK/FK e cardinalidades.
- ✓ **Físico:** tipos, índices, particionamento e tuning por SGBD.
- ✓ Ferramentas: MySQL Workbench, dbdiagram.io, draw.io, DBeaver.
- ✓ No breakout paramos no modelo lógico — base da Entrega 02.



45697056

Redundância, anomalias e normalização

- ✓ Redundância gera anomalias de **inserção**, **atualização** e **exclusão**.
- ✓ **1FN**: atributos atômicos, sem repetição.
- ✓ **2FN**: 1FN + sem dependência parcial da chave.
- ✓ **3FN**: 2FN + sem dependência transitiva.
- ✓ Aprofundamento: **material complementar 01B**.

	A	B	C	D	E	F	G	H	I
1	Produto	Pedido	Cliente	Loja	Valor	Quantidade	Data	Vendedor	
2	Lapis	1	Ana	Pinheiros	R\$ 100	20	01/01/2012	Capistrano	
3	Régua	1	Ana	Pinheiros	R\$ 120	22	01/01/2012	Capistrano	
4	Caderno	2	Claudia	Jabaquara	R\$ 140	32	31/01/2012	Arnaldo	
5	Caneta	2	Claudia	Jabaquara	R\$ 110	12	31/01/2012	Arnaldo	
6	Lapis	2	Claudia	Jabaquara	R\$ 100	34	31/01/2012	Arnaldo	
7	Régua	3	Quirino	Santana	R\$ 120	34	07/03/2012	Soares	
8	Caderno	3	Quirino	Santana	R\$ 140	23	07/03/2012	Soares	
9	Caneta	4	Beatriz	Santana	R\$ 110	23	01/04/2012	Suelen	
10	Lapis	5	Raposo	Santana	R\$ 100	67	07/05/2012	Suelen	
11	Régua	6	Dalva	Santana	R\$ 120	43	06/06/2012	Suelen	
12	Caderno	7	Claudia	Jabaquara	R\$ 140	96	17/07/2012	Arnaldo	
13	Caneta	8	Paulo	Tatuapé	R\$ 110	27	07/08/2012	Vieira	
14	Lapis	9	Quirino	Santana	R\$ 100	91	07/07/2012	Suelen	
15	Régua	10	Ana	Pinheiros	R\$ 120	35	10/10/2012	Marina	
16	Caderno	10	Ana	Pinheiros	R\$ 140	61	10/10/2012	Marina	
17	Caneta	11	Dalva	Santana	R\$ 110	83	15/11/2012	Evaldo	
18	Borracha	11	Dalva	Santana	R\$ 150	64	15/11/2012	Evaldo	
19	Borracha	11	Dalva	Santana	R\$ 150	71	15/11/2012	Evaldo	
20									

Structured Query Language (SQL)

FIAP

DDL: Data Definition Language → CREATE, ALTER, TRUNCATE, DROP, DESCRIBE, RENAME

DML: Data Manipulation Language → INSERT, UPDATE, DELETE, SELECT

DCL: Data Control Language → GRANT, REVOKE

TCL: Transaction Control Language → ROLLBACK, COMMIT

DDL na prática:

CREATE TABLE (QuantumFinance)

```
CREATE TABLE cliente (
```

```
    id_cliente    INT            PRIMARY KEY,  
    cpf           CHAR(11)       NOT NULL UNIQUE,  
    nome          VARCHAR(120)   NOT NULL,  
    data_cad      DATETIME       DEFAULT NOW() );
```

```
CREATE TABLE conta (
```

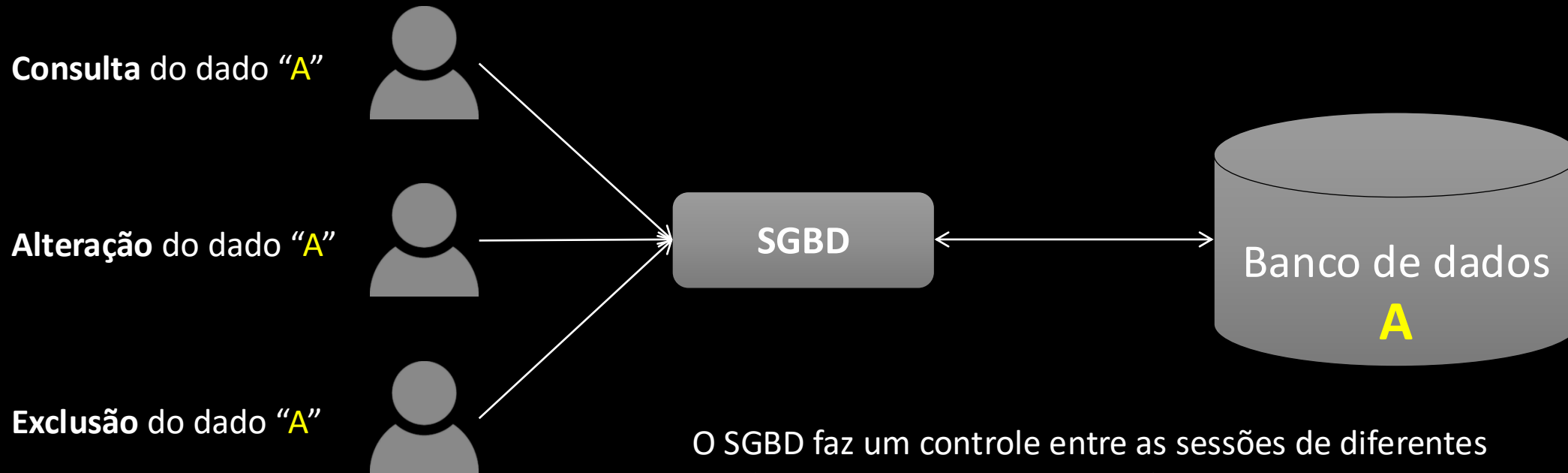
```
    id_conta      INT            PRIMARY KEY,  
    id_cliente    INT            NOT NULL,  
    tipo          VARCHAR(20)    NOT NULL,  
    saldo         DECIMAL(15,2)  DEFAULT 0,  
    FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente) );
```

```
CREATE TABLE transacao (
```

```
    id_transacao  BIGINT         PRIMARY KEY,  
    id_conta      INT            NOT NULL,  
    modalidade    ENUM('DOC','TED','PIX') NOT NULL,  
    valor         DECIMAL(15,2)  NOT NULL CHECK (valor > 0),  
    data_hora     DATETIME       NOT NULL,  
    FOREIGN KEY (id_conta) REFERENCES conta(id_conta) );
```

45697056

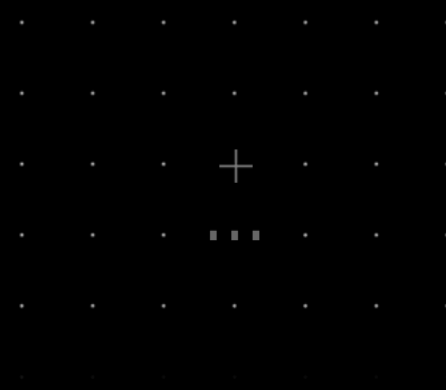
Controle de sessões



O SGBD faz um controle entre as sessões de diferentes usuários para garantir a consistência dos dados.

Há opções para consulta de dados com transações em andamento.

Metadado



Metadado de um banco de dados é o “dado do dado”

Cada sistema de banco de dados possui tabelas de sistema que controlam a especificação de cada tabela, índices, configurações, permissões, usuários, senhas, parâmetros, sessões e demais informações que o SGBD usa para otimizar sua performance.

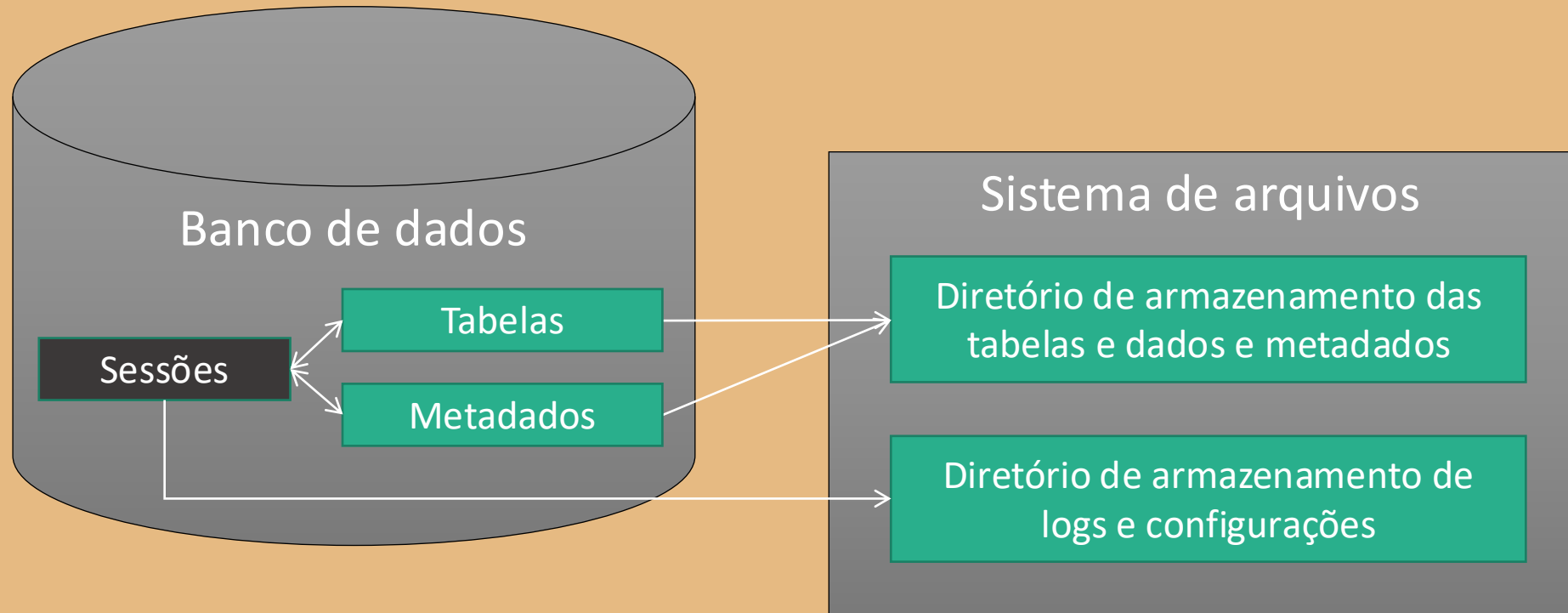
Estas tabelas são visíveis e acessíveis somente para usuários administradores

Ex.: O SGBD gerencia uma tabela interna que armazena os nomes de todas as tabelas existentes no banco

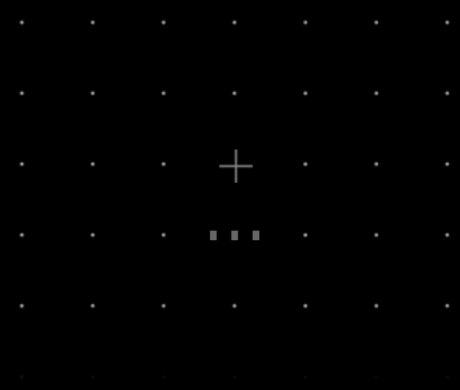
45697056

Armazenamento

Servidor



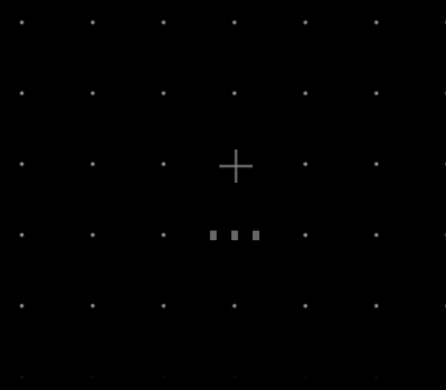
Procedures



Alguns SGBDs implementam procedimentos com regras de negócio para tratamento dos dados.

São úteis para criação de processos batch, integrações, importações, etc

Triggers



São rotinas disparadas a partir de um evento de manipulação de dados.

Disparadas de acordo com a configuração definida, como inclusão, alteração ou exclusão de um dado.

Ex: Quando um registro é inserido em uma tabela, uma trigger pode gravar um campo no registro incluído com o nome do usuário e a data da criação do registro



45697056

Transações



Transações são conjuntos de operações de manipulação de dados que são efetivas ou desfeitas em conjunto.

Caso a transação seja concluída até o fim com sucesso, todas as operações são efetivas.

Caso contrário, as operações não são salvas e os dados voltam ao estado anterior ao início da transação

É útil quando há dependência funcional entre uma ou mais operações realizadas no banco.

Ex.: Suponha que para confirmar um pagamento seja necessário alterar 3 tabelas. Uma transação garante que as 3 tabelas serão alteradas para confirmar o pagamento.

Quando usar um banco relacional

- ✓ Estrutura pré-definida de dados
- ✓ Consistência de dados mais importante do que escalabilidade e elasticidade
- ✓ Sem necessidade de consultas em tempo real
- ✓ Simplicidade e velocidade no desenvolvimento
- ✓ Acesso ao dado pelo usuário final facilitado
- ✓ Volumes de dados na casa de gigabytes ou terabytes

Exemplos de bancos relacionais

FIAP

+



+



45697056

Breakout — Modelagem de Dados (QuantumFinance)

Breakout — Contexto do caso

- ✓ QuantumFinance: fintech em expansão que precisa de uma arquitetura de dados escalável e confiável.
- ✓ O núcleo transacional (OLTP) é relacional — é o que vamos modelar agora.
- ✓ Recorte do exercício: **Cadastro de cliente + contatos** e **Transações DOC/TED/PIX**.
- ✓ Este modelo alimenta diretamente a Entrega 02 do projeto integrado (pipeline de ingestão no Apache NiFi).

Trabalho em grupo — Parte 1 (Relacional / MySQL)

A QuantumFinance precisa garantir que seus clientes realizem transações e adquiram ou liquidem produtos e investimentos com total **integridade** — de modo que o crédito seja gerado corretamente e que cada transação seja registrada com o produto, a conta e o valor certos, sem duplicidade nem inconsistência.

Como time técnico, vocês decidem sustentar essa operação em um banco relacional (**MySQL**), aproveitando integridade referencial e transações ACID.

Tarefa do grupo (3-4 pessoas):

- ✓ Modelem o diagrama entidade-relacionamento.
- ✓ Apliquem normalização até a 3FN.
- ✓ Gerem o script DDL (CREATE TABLE).

Atenção especial: chaves primárias e estrangeiras (integridade referencial); NOT NULL / UNIQUE / CHECK; cardinalidades; resolver N:N com tabela associativa.

Entrega: cole o script DDL no Web App (correção automática). Tragam o diagrama ER para a socialização.

Breakout — Solução de referência (modelo lógico)

```
cliente(id_cliente PK • cpf UQ • nome • data_nasc)
contato(id_contato PK • id_cliente FK→cliente • tipo • valor)
conta(id_conta PK • id_cliente FK→cliente • tipo • saldo • abertura)
transacao(id_transacao PK • id_conta FK→conta • modalidade • valor • data_hora)
chave_pix(id_chave PK • id_conta FK→conta • tipo_chave • valor_chave)
produto_credito(id_produto PK • nome • taxa)
cliente_produto(id_cliente FK • id_produto FK • PK composta • data_oferta) ← N:N
```

Cardinalidades: cliente 1:N conta • conta 1:N transacao • cliente 1:N contato
_____ conta 1:N chave_pix • cliente N:N produto_credito

45697056

PK em amarelo • FK em verde • UQ = UNIQUE

CHAMADA !!!



15 min.

Recapitulando...

- ✓ Fundamentos: modelo de Codd, SGBD/SGBDR, dados percebidos como tabelas.
- ✓ Integridade: PK, chave candidata/artificial, FK e integridade referencial.
- ✓ Relacionamentos e cardinalidade (1:1, 1:N, N:N) e tabela associativa.
- ✓ Modelagem conceitual → lógico → físico; redundância e normalização (1FN/2FN/3FN).
- ✓ SQL (DDL/DML/DCL/TCL) e hands-on em MySQL.
- ✓ Breakout de modelagem: caso QuantumFinance.

Bancos de dados não apenas relacionais (Not Only SQL / NoSQL)

Surgimento de bancos NoSQL

Com o crescimento nos volumes de dados gerados, tipos diferentes e escalabilidade global, a fragilidade dos bancos relacionais sobre esses aspectos favoreceram o surgimento de tecnologias capazes de entregar estas características e que ajudam a constituir o ecossistema de big data. Essas tecnologias ficaram conhecidas como Not Only SQL (NoSQL).

O Teorema de CAP

C – Consistency / Consistência

A – Availability / Disponibilidade

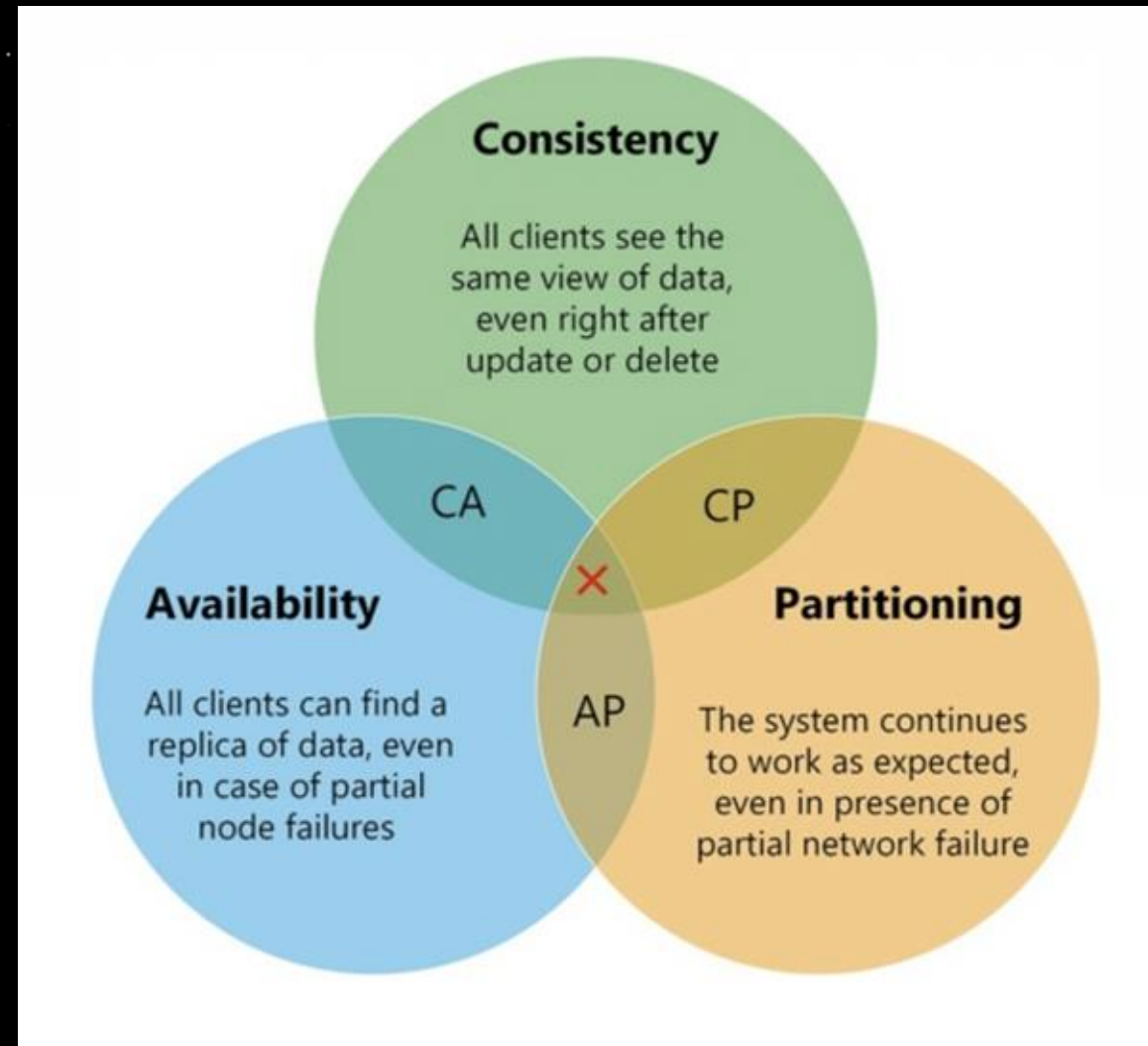
P – Partitioning / Particionamento

O Teorema de CAP afirma que um sistema distribuído de dados não é capaz de entregar simultaneamente mais de duas dessas capacidades.

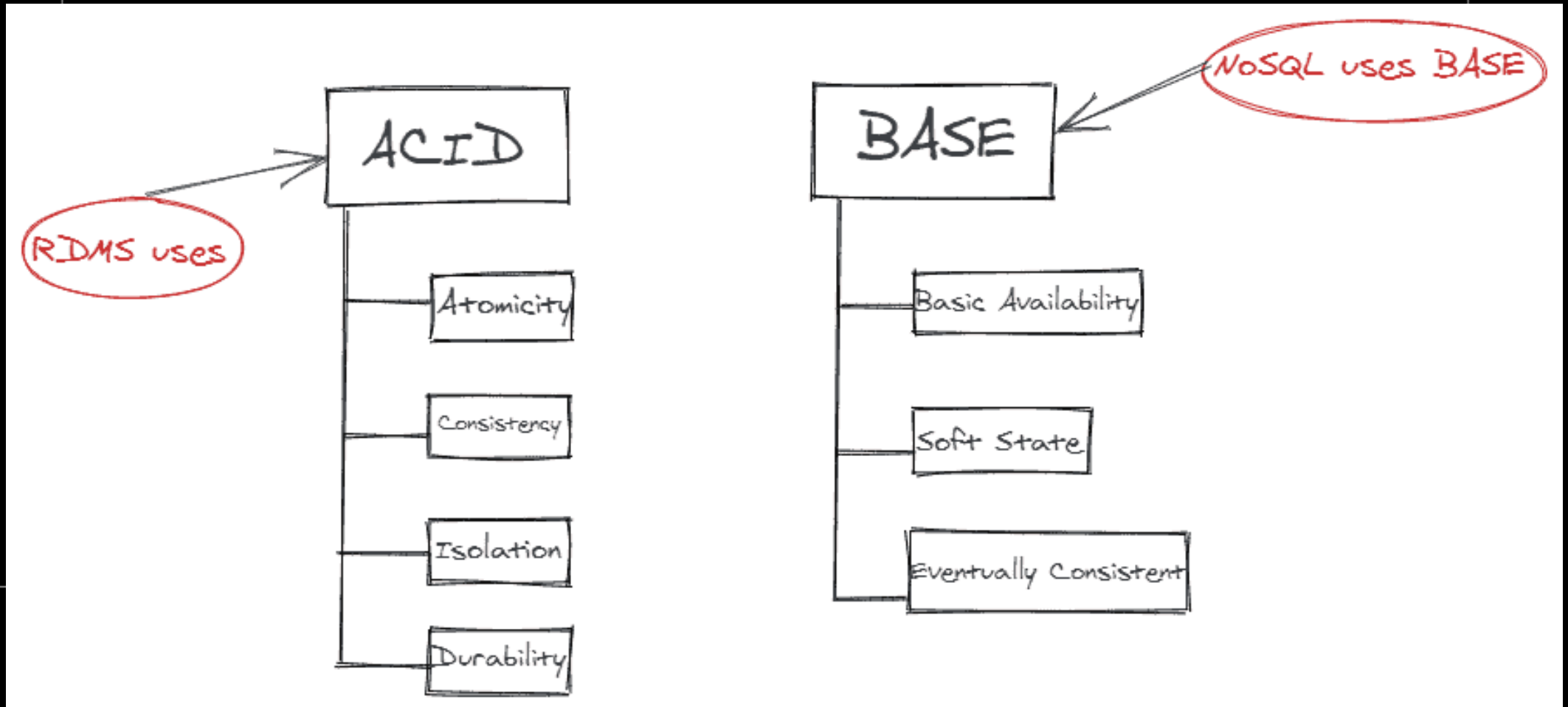
Foi criado pela primeira vez pelo professor Eric A. Brewer em 2000.

+

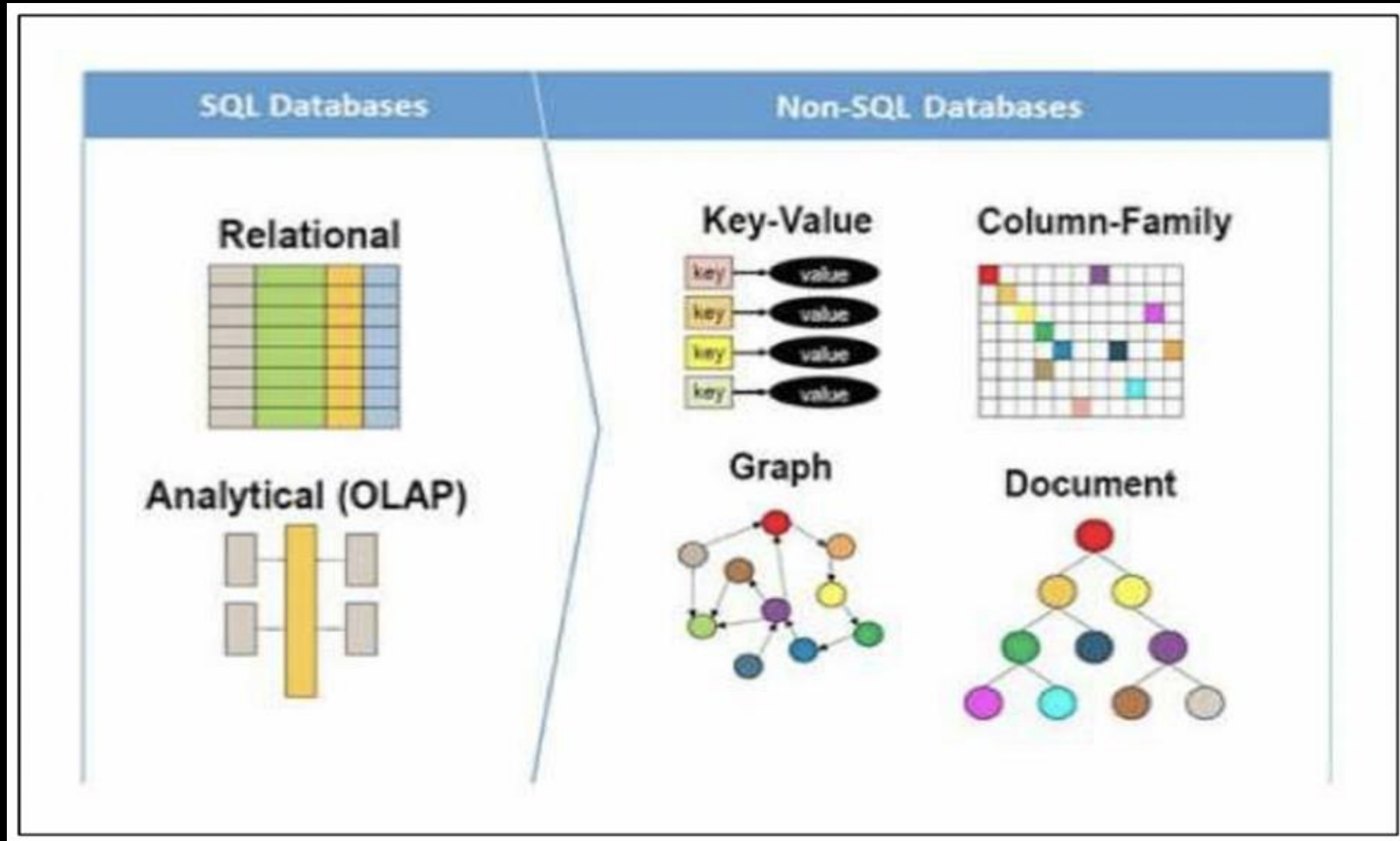
Comparativo: Barato x Rápido x Bom



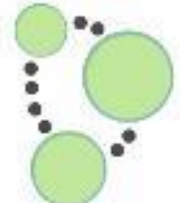
Diferenças na operação



Tipos de NoSQL



Alguns exemplos de NoSQL

Type	Example	
Key-Value Store	 redis	 riak
Wide Column Store	 APACHE HBASE	 cassandra
Document Store	 mongoDB	 CouchDB relax
Graph Store	 Neo4j	 The Distributed Graph Database

Quando usar bancos NoSQL:

- ✓ Flexibilidade
- ✓ Escalabilidade
- ✓ Disponibilidade
- ✓ Baixo custo operacional
- ✓ Sem necessidade de schema
- ✓ Funcionalidades diferenciadas

Além disso...

- ✓ Facilitam integrações por eventos
- ✓ Consultas e gravações em sub-segundo
- ✓ APIs nativas
- ✓ Open-source

Bancos colunares

DB-Engines Ranking of Wide Column Stores

The DB-Engines Ranking ranks database management systems according to their popularity. The ranking is updated monthly.

This is a partial list of the [complete ranking](#) showing only wide column stores.

Read more about the [method](#) of calculating the scores.

☐ include secondary database models

13 systems in ranking, April 2025

Rank			DBMS	Database Model	Score		
Apr 2025	Mar 2025	Apr 2024			Apr 2025	Mar 2025	Apr 2024
1.	1.	1.	Apache Cassandra	Wide column, Multi-model 	108.21	+1.55	+4.34
2.	↑3.	↑3.	Microsoft Azure Cosmos DB	Multi-model 	22.55	+0.28	-7.29
3.	↓2.	↓2.	Apache HBase	Wide column	22.12	-1.97	-9.13
4.	4.	4.	Datastax Enterprise	Wide column, Multi-model 	3.94	+0.05	-2.37
5.	5.	5.	ScyllaDB	Wide column, Multi-model 	3.70	+0.10	-1.57
6.	6.	↑8.	Google Cloud Bigtable	Multi-model 	2.67	+0.01	-0.90
7.	7.	↓6.	Microsoft Azure Table Storage	Wide column	2.63	-0.03	-2.29
8.	8.	↓7.	Apache Accumulo	Wide column	2.45	+0.02	-1.16
9.	9.	9.	Amazon Keyspaces	Wide column	0.99	+0.01	+0.07
10.	10.	10.	HPE Ezmeral Data Fabric	Multi-model 	0.87	+0.11	-0.02
11.	11.	11.	Elassandra	Wide column, Multi-model 	0.29	0.00	-0.01
12.	12.	12.	Alibaba Cloud Table Store	Wide column	0.18	0.00	-0.07
13.	13.	13.	SWC-DB	Wide column, Multi-model 	0.01	+0.00	0.00

Características:

- ✓ É possível agrupar famílias de dados para um mesmo ID
- ✓ A separação por família otimiza o tempo de consultas e gravações, melhora o particionamento
- ✓ Versionamento dos dados

Usos:

- ✓ Aplicações com alto volume de consultas com usos diferentes para o mesmo ID (ex: cadastro do cliente)
- ✓ Necessidade de versionamento (ex: séries temporais)

Benefícios e aplicações

- ✓ Velocidade de consulta e gravação de registros
- ✓ Registros com esquema flexível a cada família de coluna
- ✓ Usados para gravação logs, bases de atributos de objetos, IoT, Análise de dados em tempo real
- ✓ Demais dados que possuam uma chave de identificação e tenham um conteúdo de dados que seja variável

45697056

Banco colunar: Cassandra

O Cassandra é um banco de dados NoSQL que pertence à categoria de banco de dados da column-store NoSQL.

Ele é um projeto Apache e possui uma versão Enterprise mantida pelo DataStax.

O Cassandra é escrito em Java e é usado principalmente para dados de séries temporais, como métricas, IoT (Internet of Things), logs, mensagens de bate-papo, e assim por diante. Ele é capaz de lidar com uma enorme quantidade de gravações e leituras e escala para milhares de nós.



45697056

Banco colunar: Cassandra

Alta escalabilidade

O Cassandra é altamente escalável, o que facilita a adição de mais hardware para conectar mais clientes e mais dados, conforme a necessidade;

Arquitetura Rígida

O Cassandra não possui um único ponto de falha e está continuamente disponível para aplicativos essenciais aos negócios que não podem pagar por uma falha.

Desempenho rápido em escala linear

Cassandra é linearmente escalável. Isso aumenta sua taxa de transferência porque facilita o aumento do número de nós no cluster. Por isso, mantém um tempo de resposta rápido;



Banco colunar: Cassandra

Tolerante a falhas

Cassandra é tolerante a falhas. Suponha que haja 4 nós em um cluster, aqui cada nó tem uma cópia dos mesmos dados. Se um nó não estiver mais em serviço, outros três nós poderão ser atendidos por solicitação;

Armazenamento de dados flexível

O Cassandra suporta todos os formatos de dados possíveis, como estruturado, semiestruturado e não estruturado. Isso facilita você a fazer alterações em suas estruturas de dados de acordo com sua necessidade;

Distribuição Fácil de Dados

A distribuição de dados no Cassandra é muito fácil, pois fornece a flexibilidade de distribuir dados onde você precisa, replicando dados em vários datacenters;



Banco colunar: Cassandra

Suporte de Transação

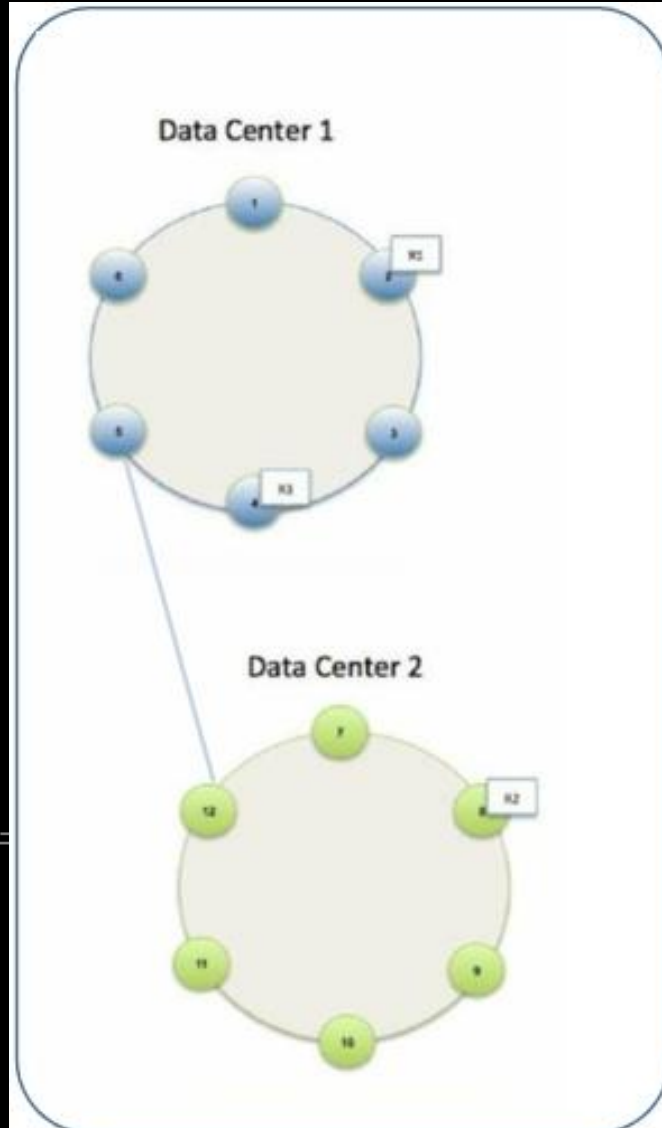
O Cassandra suporta propriedades como Atomicidade, Consistência, Isolamento e Durabilidade (ACID);

Escreve rápido

O Cassandra foi projetado para rodar em hardware barato. Ele executa gravações incrivelmente rápidas e pode armazenar centenas de terabytes de dados, sem sacrificar a eficiência de leitura.



Arquitetura Cassandra



Componentes Cassandra:

Nó - é o local onde os dados são armazenados. É o componente básico de Cassandra.

Data Center - é uma coleção de nós. Muitos nós são categorizados como um data center.

Cluster - é a coleção de muitos data centers.



Replicação de dados

Os dados são replicados para garantir nenhum ponto único de falha em caso de falha de hardware ou rede/conectividade, neste caso o Cassandra coloca réplicas de dados em diferentes nós com base nesses dois fatores:

Onde colocar a próxima réplica é determinado pela **Estratégia de Replicação**. Enquanto o número total de réplicas colocadas em nós diferentes é determinado pelo **Fator de Replicação**.

Um fator de replicação significa que há apenas uma única cópia de dados, enquanto três fatores de replicação significam que há três cópias dos dados em três nós diferentes. Para garantir que não haja um único ponto de falha, o fator de replicação deve ser três.



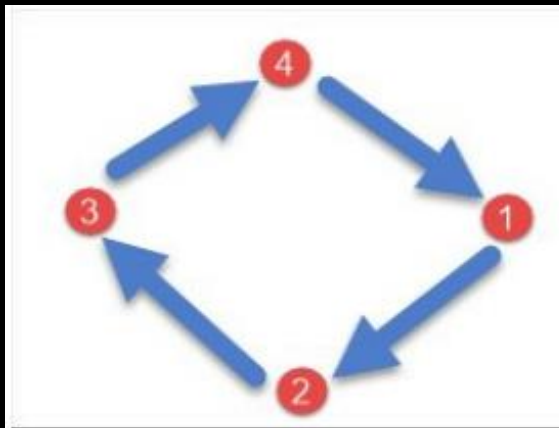
Estratégia de replicação

Existem dois tipos de estratégias de replicação no Cassandra:

SimpleStrategy

NetworkTopologyStrategy

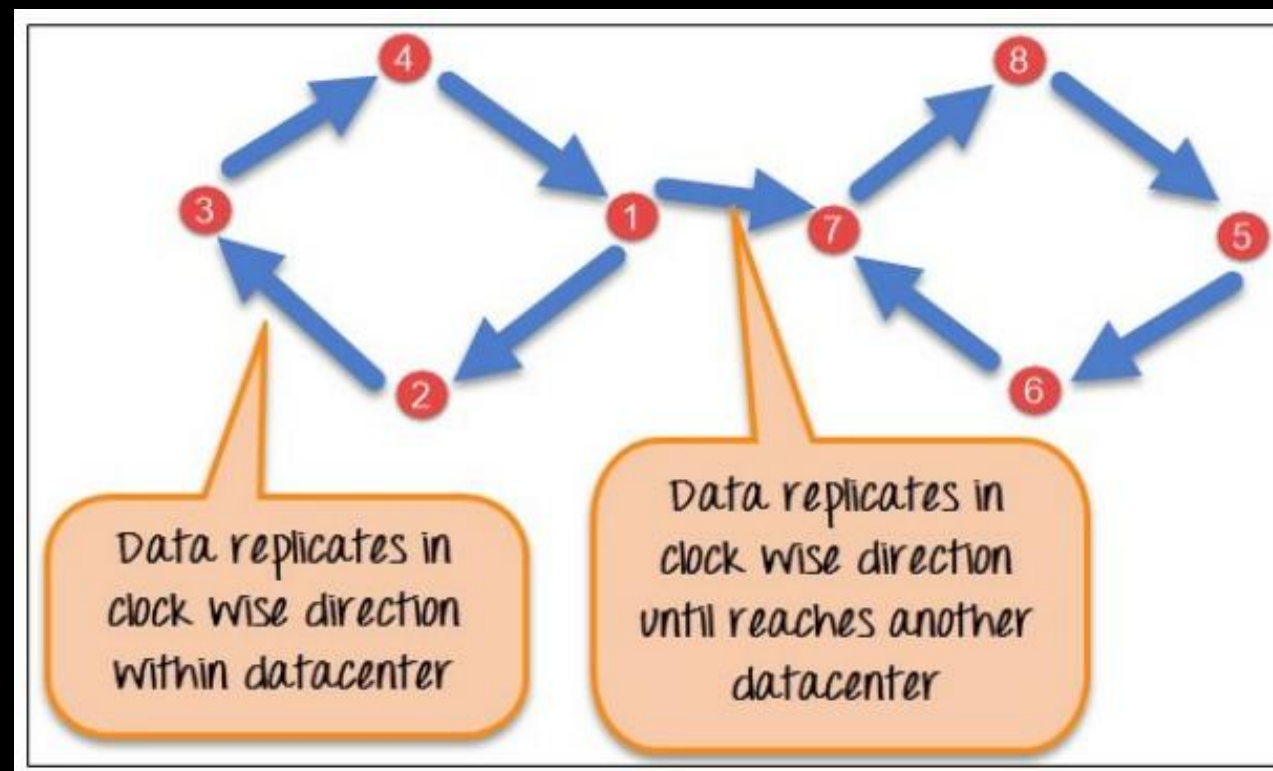
O **SimpleStrategy** é usado quando você tem apenas um data center. SimpleStrategy coloca a primeira réplica no nó selecionado pelo particionador. Depois disso, as réplicas restantes são colocadas no sentido horário no anel do nó.



Estratégia de replicação

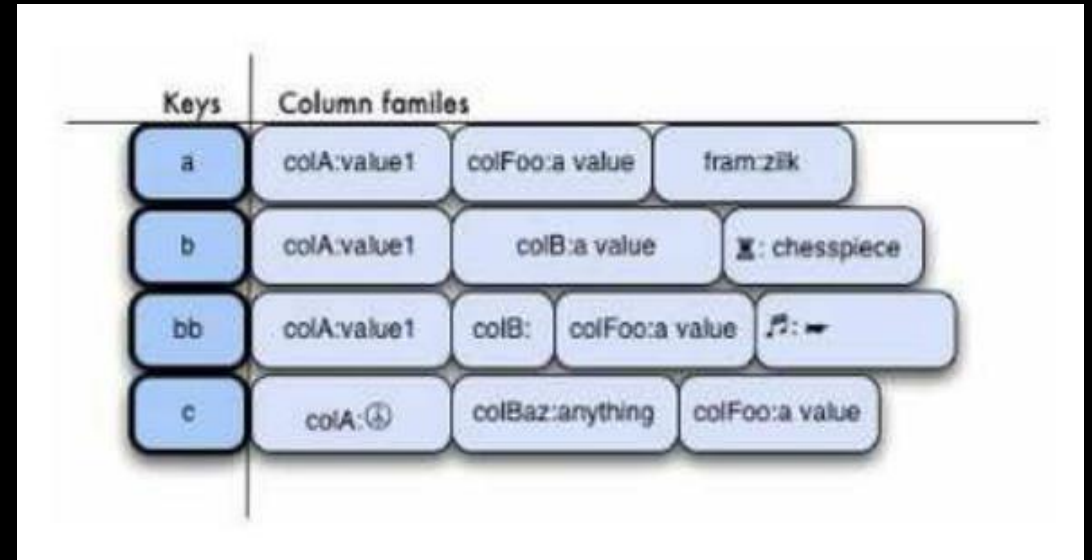
NetworkTopologyStrategy é usado quando você tem mais de dois data centers. As réplicas são definidas para cada data center separadamente. **NetworkTopologyStrategy** coloca réplicas no sentido horário no anel até atingir o primeiro nó em outro rack.

Essa estratégia tenta colocar réplicas em diferentes racks no mesmo data center. Isso se deve à razão de que, às vezes, falhas ou problemas podem ocorrer no rack. Em seguida, as réplicas em outros nós podem fornecer dados.



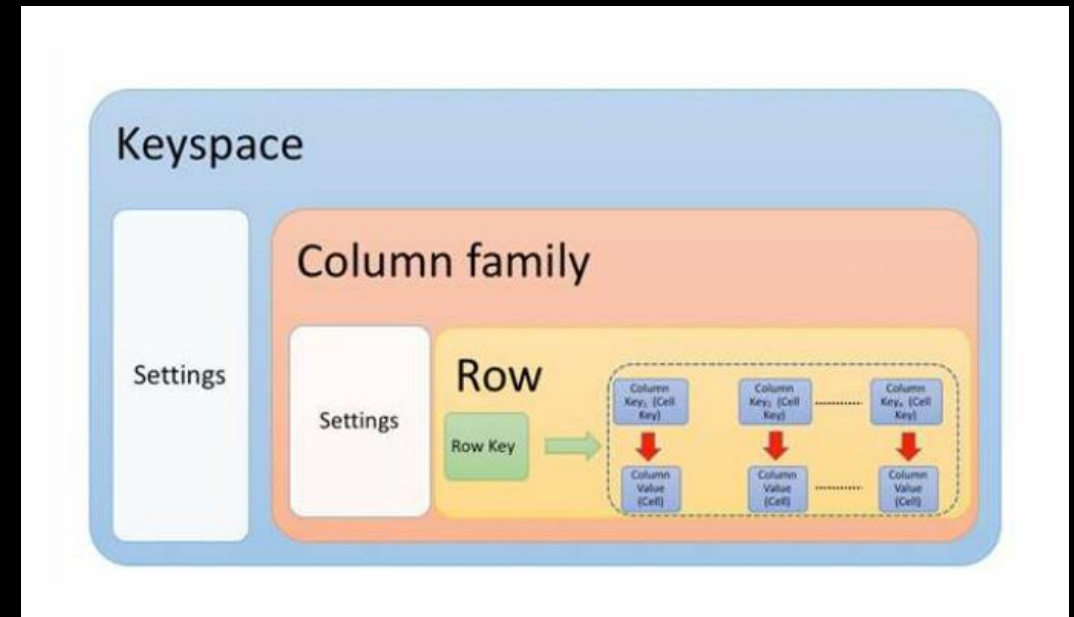
Armazenamento colunar (vs orientado a linha)

- ✓ Bancos colunares armazenam por COLUNA, não por linha — colunas iguais ficam fisicamente juntas no disco.
- ✓ Consultas analíticas que leem poucas colunas em muitos registros ficam muito mais rápidas.
- ✓ Em Cassandra, dados da mesma **família de colunas** são gravados juntos por chave — otimizando partições.
- ✓ É possível adicionar colunas a um registro sem alterar o schema da tabela (schema flexível por linha).



Organização: Keyspace, tabela e linha

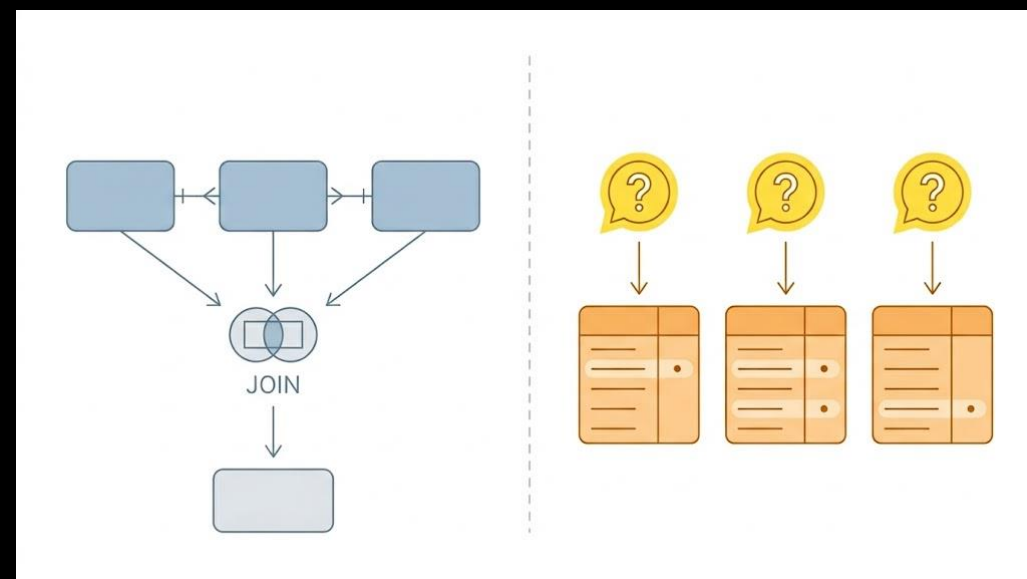
- ✓ **Keyspace**: contêiner mais externo (~database no relacional). Define fator e estratégia de replicação.
- ✓ **Tabela** (column family): estrutura dentro do keyspace; cada tabela tem seu PRIMARY KEY.
- ✓ **Linha**: identificada pela **partition key**; pode ter colunas diferentes (schema flexível).
- ✓ Lembrete: nada de JOIN entre tabelas — a modelagem resolve isso de outro jeito (próximo slide).



45697056

Modelagem em Cassandra: query-first

- ✓ Em SQL, modelamos pelas **entidades** (normalização). Em Cassandra, modelamos pelas **consultas** que precisamos atender.
- ✓ Princípio: uma tabela por padrão de consulta. Desnormaliza-se e duplica-se dados de propósito.
- ✓ Não há JOIN: o que você precisa ler junto, precisa estar gravado junto.
- ✓ Cada tabela é projetada de fora para dentro: 'quero responder a esta pergunta, com que latência?'
- ✓ Modelo errado degrada drasticamente performance e custo — refazer modelo > tentar 'consertar' com índice.



45697056

PRIMARY KEY = Partition Key + Clustering Key

- ✓ **Partition Key**: define em qual **nó** o dado mora (hash). É o primeiro elemento da PRIMARY KEY.
- ✓ **Clustering Key**: define a **ordem** dos registros dentro da partição.
- ✓ Sintaxe: **PRIMARY KEY ((partition_cols), clustering_cols)** — parênteses extras compõem a partição.
- ✓ Boa partition key: cardinalidade alta, distribui bem, sem estourar o tamanho da partição.
- ✓ Use **WITH CLUSTERING ORDER BY (col DESC)** para fixar a ordem física dos dados.

Partition Key	Clustering Key	Data Columns
Genre Band_name	player_name	creation_date
		principal_instrument
		secondary_instrument
	player_name	birth_date
		birth_country

Exemplo: Partition + Clustering em ação

- ✓ **Partition:** (Genre, Band_name) — toda a banda fica numa mesma partição (um nó).
- ✓ **Clustering:** player_name — ordena os integrantes dentro da banda.
- ✓ Consulta natural: 'todos os integrantes de Classic Rock / Beatles' = 1 partição, 1 leitura sequencial.
- ✓ Cuidado: se uma banda tiver muitos integrantes, a partição cresce — mitiga com bucket (próximo slide).
- ✓ Mesmo padrão no QuantumFinance: **PRIMARY KEY** ((id_conta, ano_mes), data_hora, id_tx).

Partition Key		Clustering Key	Data Columns	
Genre	band	player_name	birth_date	instrument
Classic Rock	Beatles, the	John Lennon	09/10/40	Vocal
		Paul McCartney	18/06/42	Baixo
		Ringo Starr	07/07/40	Bateria
Classic Rock	Deep Purple	Ian Gillan	19/08/45	Vocal
		Ritchie Blackmore	14/04/45	Guitarra
Pop Rock	Greta Van Fleet	Jake Kiszka	23/04/96	Guitarra
		Josh Kiszka	23/04/96	Vocal

Bucketing, wide partitions e anti-padrões

- ✓ **Wide partition**: partição cresce sem limite (ex.: 5 anos de transações de uma conta) → leitura lenta, hotspot.
- ✓ Mitigação: **bucket temporal** na partition key — ano_mes, dia, semana — limita o tamanho.
- ✓ Regra prática: ≤ 100 MB ou $\leq \sim 100$ mil linhas por partição.
- ✓ **ALLOW FILTERING** e **índice secundário**: tapa-buracos que falham em escala. Re-modele em vez de usar.
- ✓ Tentar 'normalizar como SQL' em Cassandra é o erro mais comum — duplique e modele por consulta.

COM BUCKET POR MÊS — partições limitadas				
PRIMARY KEY ((id_conta, ano_mes), data_hora, id_tx)				
id_conta	ano_mes	data_hora	modalidade	valor
91234	2024-01	2024-01-02 13:09	TED	R\$ 3,272.13
91234	2024-01	2024-01-06 09:52	DOC	R\$ 515.94
91234	2024-01	2024-01-10 17:03	DOC	R\$ 1,112.76
91234	2024-01	2024-01-14 09:27	TED	R\$ 395.78
91234	2024-01	2024-01-18 09:35	TED	R\$ 342.60
91234	2024-01	2024-01-22 17:07	PIX	R\$ 3,171.60
91234	2024-02	2024-02-02 17:03	DOC	R\$ 2,948.43
91234	2024-02	2024-02-06 08:14	PIX	R\$ 2,805.49
91234	2024-02	2024-02-10 10:18	TED	R\$ 764.06
91234	2024-02	2024-02-14 09:36	TED	R\$ 2,823.27
91234	2024-02	2024-02-18 18:11	PIX	R\$ 2,928.92
91234	2024-02	2024-02-22 18:12	TED	R\$ 532.28
91234	2024-03	2024-03-02 19:04	DOC	R\$ 345.03
91234	2024-03	2024-03-06 11:31	DOC	R\$ 2,682.02
91234	2024-03	2024-03-10 20:20	TED	R\$ 2,948.53
91234	2024-03	2024-03-14 15:23	TED	R\$ 1,279.71
91234	2024-03	2024-03-18 10:44	PIX	R\$ 455.18
91234	2024-03	2024-03-22 12:33	TED	R\$ 4,381.93

Breakout — Modelagem de Dados (QuantumFinance)

Trabalho em grupo — Parte 2 (Cassandra / colunar)

Com a QuantumFinance em escala, o volume de transações (PIX, TED, DOC) passou a milhões por dia e o time de produto precisa de leituras em **subsegundo** para os painéis — algo que o modelo relacional, com joins e índices, não sustenta horizontalmente.

Como time técnico, vocês decidem guardar o histórico de transações em um banco colunar distribuído (**Apache Cassandra**), modelando a partir das consultas (query-first).

O modelo precisa responder a estas perguntas de negócio:

- ✓ Quais foram as últimas transações de uma conta, da mais recente para a mais antiga?
- ✓ Quais transações de uma conta ocorreram em um determinado mês?
- ✓ Qual o total transacionado por modalidade (PIX/TED/DOC) em um dia?

Tarefa: decidam quantas tabelas criar e justifiquem; definam **partition key** e **clustering columns** de cada tabela; avaliem hotspots e mitiguem com bucket; gerem o script CQL.

Atenção especial: PRIMARY KEY (partição + clustering), CLUSTERING ORDER e dimensionamento da partição.

Entrega: cole o CQL no Web App; informe quantas tabelas e por quê. Tragam o modelo para a socialização.

45697056

Entrega do trabalho

Onde entregar:

Aplicativo Web (correção automática)

<https://tinyurl.com/2ydc9rw8>

Como funciona:

- ✓ Identifique o grupo e os integrantes.
- ✓ Cole o script DDL (Parte 1) e/ou o CQL (Parte 2).
- ✓ Para a Parte 2, informe quantas tabelas o grupo modelou.
- ✓ Receba a nota e o feedback critério a critério na hora.
- ✓ Podem reenviar quantas vezes precisarem.



45697056

Exercício adicional

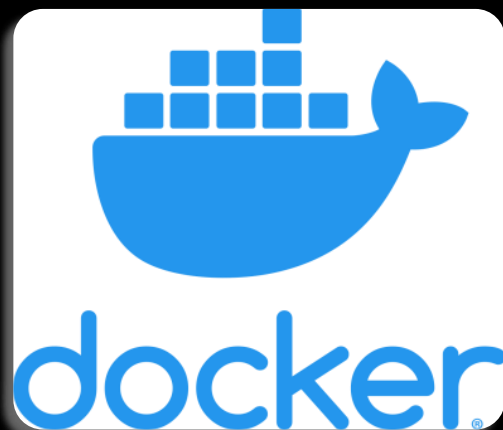
Este exercício ilustra a criação de chaves complexas no Cassandra

<https://gist.github.com/leandrommendes/b1dcf5eb48d9188758d49df6e42c27da>

Questão

Quais foram as diferenças entre fazer estes exercício usando um banco relacional e usando um banco colunar?

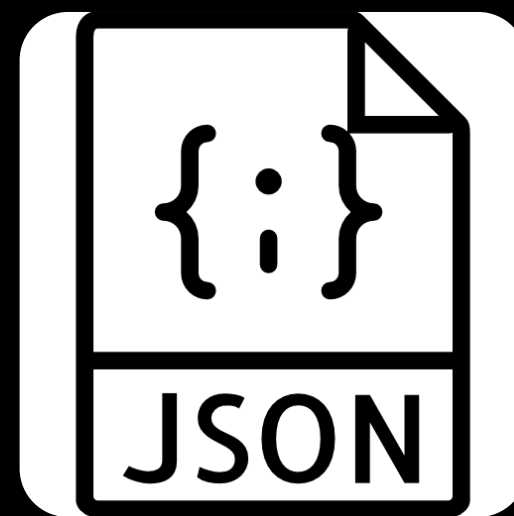
Na próxima aula para os exercícios iremos
usar:



<https://www.docker.com/products/docker-desktop/>

Repos

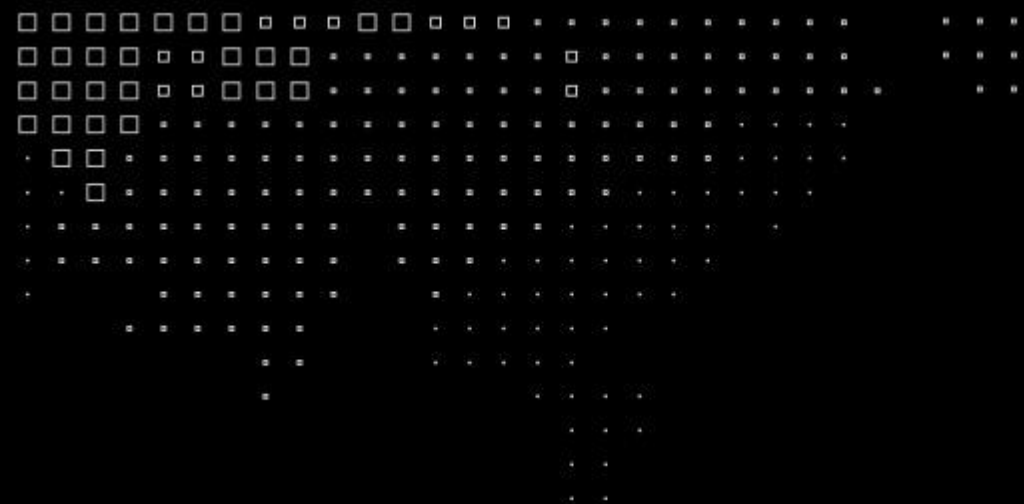
ivangancev/mongo:latest
neo4j:latest



<https://docs.fileformat.com/web/json/>

CHAMADA !!!

Respondam o Experience Survey



MBA⁺

Copyright © 2026 Prof. Leandro Mendes

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor Leandro Mendes

[Material baseado em conteúdo anterior da disciplina \(FIAP\).](#)

